DevSync

Secure AI Project Collaborator

Requirements Gathering

Project Type	Graduation Project (Flutter mobile application)
Team	6 members, all Flutter developers
Project Manager	Mohraeel (Project Manager and Flutter Lead)
Platform	Android (Android App Bundle), Flutter + Firebase
Document Version	1.1 - October 2026

Contents: 1.1 Stakeholder Analysis | 1.2 User Stories | 1.3 Use Cases | 1.4 Functional Requirements | 1.5 Non-Functional Requirements | 1.6 Constraints and Assumptions

1. Requirements Gathering

1.1 Stakeholder Analysis

Stakeholder	Role	Needs and Expectations	Influence	Interest
Developers and contributors	Primary users	Find projects that match their skills; understand why; learn what to improve; join teams easily	High	High
Beginners and students	Primary users	Beginner-friendly projects, clear skill gaps, safe first collaboration	Medium	High
Project owners and maintainers	Primary users	Find contributors with the right skills; discover missing roles; manage requests, tasks and the team	High	High
Lecturer and evaluators	Evaluates the project	Complete documentation, correct use of the course technologies, working demo, security and Git practices	High	High
Development team	Builds the system	Clear scope, realistic plan, maintainable architecture, fair task split	High	High
External services (GitHub, Firebase, LLM provider, Google Play)	Platform dependencies	Usage within API limits and terms; secure handling of tokens and user data	Medium	Low
Open-source community	Indirect beneficiary	More contributors and healthier projects	Low	Medium

1.2 User Stories

Feature numbers refer to the team's agreed feature list. Priority follows the MoSCoW approach in the project plan.

ID	User Story	Priority	Acceptance Criteria
US-01	As a developer, I want to sign in with Google or email so that I can start quickly and securely.	**Must**	User can sign in with either method; a new user is taken to profile setup; the session persists after restart.
US-02	As a developer, I want to enter my name, level, main track, education, languages, LinkedIn and portfolio, and to have a unique user ID in the app, so that matching reflects who I am and others can identify me.	**Must**	Required fields are validated; a unique user ID is created for each user and shown on the profile; data is saved and editable later.
US-03	As a developer, I want to connect my GitHub account so that my skills are detected from my real activity.	**Must**	Repositories, languages and activity are imported; if import fails, the user can continue with manual data.
US-04	As a developer, I want to add other links such as Behance so that my profile shows my full work.	**Should**	Links are saved, validated and shown on the profile.
US-05	As a developer, I want to search and filter projects by language, level and needed role so that I can explore options.	**Must**	Filters work together; results appear within 5 seconds.
US-06	As a developer, I want AI-recommended projects so that I do not have to search manually.	**Must**	A ranked list with a match score is shown based on my profile.
US-07	As a developer, I want to know why a project was recommended so that I can trust the result.	**Must**	Each recommendation shows an explanation that references my matched skills.
US-08	As a developer, I want to see which of my skills match a project so that I can judge my fit.	**Must**	Matched and unmatched skills are listed for each project.
US-09	As a beginner, I want to see my skill gaps for a project so that I know what to learn.	**Must**	Missing skills are listed with a short learning suggestion.

ID	User Story	Priority	Acceptance Criteria
US-10	As a project owner, I want to create a project with description, tech stack, needed roles, difficulty and project type so that others can find it.	**Must**	Project is saved with its difficulty and type and appears in discovery.
US-11	As a project owner, I want AI to help set up my project so that I save time.	**Should**	AI suggests structure and tech stack from my description; I can edit it.
US-12	As a project owner, I want AI to generate the roles my project needs so that I build a balanced team.	**Should**	A role list is generated and editable.
US-13	As a project owner, I want an alert when a role is missing so that I can recruit for it.	**Should**	Owner sees an alert listing roles with no member.
US-14	As a developer, I want to send a collaboration request to a project so that I can join it.	**Must**	Request is sent once; its status is visible to me.
US-15	As a project owner, I want to accept or decline requests so that I control my team.	**Must**	Accepting adds the user to the team; both users are notified.
US-16	As a user, I want real-time notifications for requests, invitations, tasks, messages and project updates so that I do not miss anything.	**Must**	Push notification arrives within 10 seconds in foreground, background and terminated states.
US-17	As a project owner, I want to invite people by link or direct invite so that I can grow my team.	**Should**	Invite link opens the project and lets the invitee join; if the invitee is not signed in, they sign in or sign up and return to the same project.
US-18	As a team member, I want to create and track tasks so that the team stays organized.	**Must**	Tasks can be created, assigned, updated and completed.
US-19	As a team member, I want 1-1 and team chat so that we can communicate inside the app.	**Should**	Messages are delivered in real time and stored.
US-20	As a team member, I want a shared workspace so that project information is in one place.	**Should**	Workspace shows project details, members, tasks and links.
US-21	As a user, I want to save and favorite projects and see my match history so that I can return to them.	**Could**	Saved, favorite and past matches are listed.
US-22	As a visitor, I want to browse without an account so that I can try the app first.	**Could**	Guest can browse but must sign in to request or create; after signing in the user returns to the screen they came from.
US-23	As a user, I want a Free and a Pro version so that I can use basic features free and unlock more with Pro.	**Must**	Two flavors build and run with different feature sets.
US-24	As a user, I want a community area so that I can meet other developers.	**Could**	Users can view and post in the community area.
US-25	As a team (and as a user), we want recommendation feedback and prompt evaluation so that matching quality improves over time.	**Must**	Users can rate recommendations; ratings are stored and used to compare prompt versions.

1.3 Use Cases

UC-01: Sign in and complete profile

Actor	Developer
Precondition	App is installed and the device is online

Main flow	1. The user opens the app and chooses Google or Email sign-in. 2. The system authenticates the user through Firebase Auth. 3. For a new user, the system opens profile setup. 4. The user enters the manual profile data and optionally connects GitHub. 5. The system imports GitHub data and saves the profile.
Alternative flows	A1. Authentication fails: the system shows an error and lets the user retry. A2. GitHub import fails or is skipped: the system continues with manual data only.
Postcondition	The user has a saved profile and can use discovery

UC-02: Get AI project recommendations

Actor	Developer
Precondition	The user is signed in and has a profile
Main flow	1. The user opens project discovery. 2. The system sends the profile data to the AI matching service. 3. The system returns ranked projects with match scores. 4. The user opens a project and sees the explanation, matched skills and skill gap. 5. The user can rate the recommendation.
Alternative flows	A1. AI service is slow or unavailable: the system shows cached or rule-based results with a notice. A2. Profile is incomplete: the system asks the user to complete it first.
Postcondition	The user has seen recommendations; ratings are stored for prompt evaluation

UC-03: Send and respond to a collaboration request

Actors	Developer, Project owner
Precondition	Both users are signed in; the project is open to requests
Main flow	1. The developer opens a project and sends a collaboration request. 2. The system stores the request and sends a push notification (FCM) to the owner. 3. The owner opens the request and accepts it. 4. The system adds the developer to the team and notifies the developer.
Alternative flows	A1. The owner declines: the developer is notified and the request is closed. A2. The developer has already sent a request: the system blocks a duplicate.
Postcondition	The request has a final status; accepted users are team members

UC-04: Create a project and generate roles

Actor	Project owner
Precondition	The owner is signed in
Main flow	1. The owner enters the project title, description and tech stack. 2. The owner asks AI to suggest the setup and the needed roles. 3. The system returns suggested structure and roles; the owner edits them. 4. The owner saves the project; the system publishes it to discovery. 5. The system flags any role with no member as missing.
Alternative flows	A1. AI service fails: the owner continues by entering roles manually.
Postcondition	The project is saved with its roles and visible to other users

UC-05: Manage project tasks

Actor	Team member
Precondition	The user is a member of the project team
Main flow	1. The user opens the project workspace and the task list. 2. The user creates a task and assigns it to a member. 3. The assignee receives a notification and updates the status. 4. The team sees the updated progress.
Alternative flows	A1. The user is offline: the action is queued or the user is told to reconnect.

Postcondition	Task data is saved and visible to the whole team

1.4 Functional Requirements

ID	Module	Requirement	Priority
FR-01	Authentication	The system shall let users sign up and sign in with Google and with email and password using Firebase UI Auth.	**Must**
FR-02	Authentication	The system shall keep the user signed in across app restarts and allow sign out.	**Must**
FR-03	Profile	The system shall let users create and edit a profile with name, level, main track, education, programming languages, LinkedIn and portfolio, and shall assign each user a unique user ID in the app.	**Must**
FR-04	Profile	The system shall let users add other links (GitHub, Behance).	**Should**
FR-05	Profile	The system shall import GitHub data (repositories, languages, activity) for the connected account.	**Must**
FR-06	Discovery	The system shall let users browse, search and filter projects.	**Must**
FR-07	AI Matching	The system shall recommend projects ranked by a match score computed from the user's profile and GitHub data.	**Must**
FR-08	AI Matching	The system shall explain each recommendation in natural language.	**Must**
FR-09	AI Matching	The system shall show matched and unmatched skills between a user and a project.	**Must**
FR-10	AI Matching	The system shall show the user's skill gaps for a project.	**Must**
FR-11	AI Matching	The system shall collect user ratings of recommendations and support comparing prompt versions (prompt evaluation).	**Must**
FR-12	Projects	The system shall let users create, edit and publish projects with description, tech stack, needed roles, difficulty level and project type.	**Must**
FR-13	Projects	The system shall use AI to suggest project setup and to generate needed roles.	**Should**
FR-14	Projects	The system shall alert the owner when a required role has no member.	**Should**
FR-15	Collaboration	The system shall let users send collaboration requests and let owners accept or decline them.	**Must**
FR-16	Collaboration	The system shall support team invites and shareable project links, and return the user to the linked project after sign-in or sign-up.	**Should**
FR-17	Notifications	The system shall send push notifications (FCM) for collaboration requests, request decisions, team invitations, task assignments and updates, new team messages and project updates.	**Must**
FR-18	Tasks	The system shall let team members create, assign, update and complete tasks.	**Must**
FR-19	Communication	The system shall provide 1-1 chat and team chat.	**Should**
FR-20	Communication	The system shall provide a project workspace showing project details, members, tasks and links.	**Should**
FR-21	Utilities	The system shall let users save and favorite projects and view match history.	**Could**
FR-22	Utilities	The system shall provide a guest mode with read-only browsing.	**Could**
FR-23	Release	The system shall build in Free and Pro flavors with different feature sets.	**Must**
FR-24	Animation	The system shall use implicit and advanced animations (Tween and Transition widgets) for a smooth interface.	**Should**
FR-25	Communication	The system shall provide a community area where users can view and post.	**Could**

1.5 Non-Functional Requirements

ID	Category	Requirement	Measure
NFR-01	Performance	Recommendations shall load quickly.	5 seconds or less on a normal connection
NFR-02	Performance	Push notifications shall be delivered promptly.	10 seconds or less in tests on real devices
NFR-03	Security	Data access shall be restricted per user through Firestore security rules.	100% of the security review checklist passed
NFR-04	Security	Secrets and API keys shall not be stored in the app code or the repository.	No keys found in the repository review
NFR-05	Security	The app shall follow the Android security model and request runtime permissions only when needed.	Permission requests reviewed in testing
NFR-06	Security	Release builds shall use code obfuscation.	Release AAB built with obfuscation enabled
NFR-07	Privacy	The app shall use only the GitHub data needed for matching and let users disconnect it.	Disconnect option available and tested
NFR-08	Usability	Core tasks (sign in, find a project, send a request) shall be easy to complete.	Test users complete them without help
NFR-09	Usability	The interface shall be consistent and use smooth animations.	Shared theme and widget kit used across screens
NFR-10	Reliability	The app shall be stable and handle network errors gracefully.	95% or higher crash-free sessions; clear error messages
NFR-11	Reliability	AI features shall degrade gracefully when the AI service fails.	Fallback results shown instead of a blank screen
NFR-12	Compatibility	The app shall run on current Android devices and screen sizes.	Tested on at least two real devices
NFR-13	Maintainability	Code shall follow the agreed architecture, Git branching and pull-request review process.	100% of merges reviewed
NFR-14	Documentation	The project shall include technical documentation and a best-practice README.	Complete at the final milestone

1.6 Constraints and Assumptions

- The platform is Android only; the release format is an Android App Bundle.
- The team has six Flutter developers and a delivery window of about nine weeks (5 October to 4 December 2026), so only Must features are guaranteed.
- GitHub and the AI provider have API limits; the design uses caching and fallbacks to stay within them.
- The Pro flavor gates features only; real payment processing is out of scope.